02 / PYTHON REFERENCE GUIDE

NumPy tools for 2D rotations

EX29 · Functions, inputs, outputs and independent examples

Goal
Connect the Mathematical Theory Guide to specific Python instructions. For each function, explain what it accepts, what it returns and when to use it.

1. `np.deg2rad()`: degrees to radians

Input: a numeric angle in degrees, or an array of angles. **Output:** the corresponding angle(s) in radians. NumPy trigonometric functions expect radians.

```
import numpy as np
angle_deg = 90.0
angle_rad = np.deg2rad(angle_deg)
print(angle_rad)  # approximately 1.5707963268
```

Use `np.rad2deg()` to convert back to degrees, as in EX26. Recall that $180° = \pi$ radians.

2. `np.cos()` and `np.sin()`

Input: a **radian** angle, as a scalar or array. **Output:** cosine/sine, as a floating-point number or an array depending on the input.

```
angle_rad = np.deg2rad(90.0)
c = np.cos(angle_rad)
s = np.sin(angle_rad)
print(c)  # approximately 6.12e-17, numerically zero
print(s)  # 1.0
```

6.12×10^{-17} is not a geometric discrepancy: it is caused by finite-precision `float` arithmetic. Do not demand exact equality to zero.

3. `np.array()`: building a matrix

Input: nested rows. **Output:** a two-dimensional NumPy array. The `shape` of a 2×2 matrix is `(2, 2)`. Independent example:

```
angle = np.deg2rad(45.0)
c, s = np.cos(angle), np.sin(angle)
R = np.array([[c, -s], [s, c]])
print(R.shape)  # (2, 2)
```

One column contains the image of the first Cartesian unit vector, the other contains the image of the second.

4. `@`: matrix-vector product

Inputs: arrays with compatible dimensions, such as a `(2, 2)` matrix and a `(2,)` vector. **Output:** a `(2,)` array containing the transformed components.

```
vector = np.array([1.0, 0.0])
new_vector = R @ vector
print(new_vector)  # approximately [0.70710678, 0.70710678]
```

`R * vector` is **not** the same: NumPy performs elementwise multiplication with broadcasting.

5. `.T`: transpose of a matrix

Input: a two-dimensional array. **Output:** its transpose (generally a view). For orthogonal rotation matrices, `R.T` performs the inverse transformation.

```
vector_back = R.T @ new_vector
print(vector_back)  # approximately [1.0, 0.0]
```

Transposing a one-dimensional `(2,)` array does not turn it into a column matrix. This distinction will be revisited later.

6. `np.eye()`, `np.allclose()` and norms

`np.eye(2)` creates a 2×2 identity matrix. `np.allclose(a,b)` returns one boolean indicating whether arrays are numerically close within tolerances; unlike `np.isclose()`, it does not return an array of booleans.

```
identity = np.eye(2)
print(np.allclose(R.T @ R, identity))  # True
print(np.allclose(vector_back, vector))  # True
print(np.linalg.norm(vector))           # 1.0
```

7. Tool selection summary

Tool	When to use it
`np.deg2rad(a)`	When an input angle is given in degrees.
`np.sin(a), np.cos(a)`	To calculate trigonometric entries in R, with a in radians.
`R @ v`	To rotate a vector actively.
`R.T @ v`	To express components in rotated axes (EX29 convention).
`np.allclose()`	To verify numerical results, not exact equality.
`np.linalg.norm(v)`	To check preservation of length.

Common pitfalls
Mixing degrees and radians, confusing `*` with `@`, using R instead of R^T when the axes rotate, and demanding that floating-point results be exactly zero.